

updateAsset() — How It Works

DSA612S Assignment 1 · Question 1 · Library client & REST API

1. In one sentence

updateAsset() is the console-client feature for menu option 5. It asks for an asset tag, fetches and shows the current asset (**GET**), collects only the fields the user actually wants to change, and sends them as a partial update (**PATCH**) to the Ballerina REST service. The service applies them to the in-memory store under a lock.

2. Where it lives and who calls it

File: question1-library-client/main.bal (line ~204). It is a client-side function, not a server resource.

```
main()                                // REPL loop: printMenu -> readln
+-- dispatch(choice)                  // match "5" => return updateAsset();
  +-- updateAsset()                   // returns error?
    |-- libraryApi->get("/assets/{tag}")
    |   --HTTP GET--> get assets/[assetTag] -> getAsset()
  +-- libraryApi->patch("/assets/{tag}", changes)
    --HTTP PATCH-> patch assets/[assetTag] -> patchAsset()
<-- error? bubbles back to main() -> report(error) -> pause()
```

libraryApi is a module-level final http:Client created from the configurable apiUrl (default http://localhost:8080/library), so the full URLs are GET/PATCH http://localhost:8080/library/assets/{tag}.

3. Pseudocode — client side

```
FUNCTION updateAsset() RETURNS error OR nothing
  tag <- readRequired("Asset tag to update")    // loops until non-blank, trimmed
  current <- HTTP GET /assets/{urlEncode(tag)}  // CHECK: on any error -> return it
  display current asset
  print "Leave a field blank to keep its current value."

  changes <- empty map<string,string>
  FOR field IN [name, description, site]
    value <- trim(readLine("New " + field))
    IF value != "" THEN changes[field] <- value // addIfPresent
  status <- pickFrom(ASSET_STATUSES, optional = TRUE) // numbered menu, re-asks
  IF status chosen THEN changes["status"] <- status // until 1..5 or blank

  IF changes is empty THEN
    print "Nothing to change."; RETURN // no request sent
  updated <- HTTP PATCH /assets/{urlEncode(tag)} BODY changes (JSON) // CHECK
  print "Asset updated."; display updated
END
```

4. Pseudocode — server side (what the PATCH triggers)

service.bal resource → store.bal patchAsset():

```
RESOURCE PATCH /library/assets/{assetTag} (body bound to AssetPatch)
  // Binding: AssetPatch is a CLOSED record of optional fields
  // {name?, description?, institution?, site?, status?: AssetStatus, dateAcquired?}
  // unknown field or status not in the enum -> automatic 400 Bad Request
  result <- patchAsset(assetTag, patch)
  RETURN result IF Asset ELSE toApiError(result)

FUNCTION patchAsset(assetTag, patch) // isolated
  // --- validation OUTSIDE the lock ---
  IF patch.dateAcquired present AND not a valid YYYY-MM-DD -> ValidationError
  IF patch.institution present AND not registered -> ValidationError
  LOCK // guards isolated map<Asset> assetStore
    found <- assetStore[assetTag] // reference to the stored record
    IF found is nil -> NotFoundError
    changes <- clone(patch)
```

```

    FOR each field present in changes: found.field <- changes.field // in-place
    RETURN clone(found) // caller gets a copy, not the live record
END LOCK

```

5. How it interacts with the "database"

There is no real database. The store is an in-memory, module-level variable:

```
isolated map<Asset> assetStore = {}; // key = assetTag, 0(1) lookup
```

- **Read (GET):** `getAsset ( )` locks, looks the tag up, and returns a **clone**.
- **Write (PATCH):** `assetStore[assetTag]` returns a *reference* to the stored record (records are mutable reference values in Ballerina), so assigning `found.name = ...` updates the store directly. No put-back is needed.
- Only the six patchable fields can change. `assetTag`, components, schedules, work orders and the current loan are never touched by PATCH, so the key can't be changed this way.
- Data is lost when the service restarts. It is re-seeded on startup through `addAsset ( )`.

6. Exception / error handling

Client. Each check means: *if this is an error, stop here and return it*. The error travels `updateAsset` → `dispatch` → `main`, where `report (e)` turns it into one readable line, so the program never crashes and the menu loop continues.

Situation	What the client gets	What the user sees
Tag doesn't exist	GET returns 404 → <code>http:ApplicationResponseError</code>	! Error 404: No asset found with tag 'X' (PATCH is never sent)
Backend not running	<code>http:ClientConnectorError</code>	! Cannot reach the API at <code>http://localhost:8080/library ...</code>
Server rejects PATCH	400 / 404 → <code>ApplicationResponseError</code>	! Error 400: <message from body>
Response can't bind to Asset	data-binding error	! <error message>

Server. The store returns *distinct* typed errors. The service maps them to HTTP status codes in `toApiError ( )` with a uniform body `{code, message, resource}`:

Store error	HTTP status	code
<code>NotFoundError</code>	404 Not Found	NOT_FOUND
<code>ValidationError</code>	400 Bad Request	VALIDATION_FAILED
<code>ConflictError</code>	409 Conflict	CONFLICT (not raised by <code>patchAsset</code>)
anything else	500 Internal Server Error	INTERNAL_ERROR (logged; no stack trace leaked)

7. Concurrency

- The HTTP listener serves requests concurrently. `assetStore` is declared `isolated`, so the compiler **forces** every access to sit inside a lock block. Two PATCHes on the same asset can't interleave inside `patchAsset`. Each one applies all of its fields atomically.
- **clone()** at the lock boundary: the patch is cloned going in, and the result is cloned coming out. Nothing outside the lock holds a live reference to stored data. This is required for isolated functions.
- Validation (date format, institution registered) runs **before** the lock, so the lock is held as briefly as possible. `institutionIsRegistered` takes its own lock on `institutionStore`.
- **Not protected: lost updates across the GET→PATCH gap.** The client shows the asset, the user types for a while, and only then is the PATCH sent. Another client may have changed the asset in between. The result is last-writer-wins per field. Sending only the changed fields (PATCH, not PUT) limits the damage, but there is no `version/ETag/If-Match` check.

- Small race: an institution could be removed between the pre-lock check and the lock. That only matters for institution patches, which this client never sends.

8. Checks and validation

Where	Check	Effect
Client	<code>readRequired()</code> : tag must be non-blank (trimmed)	Re-prompts until valid
Client	<code>addIfPresent()</code> : trims input and skips blanks	Blank = 'keep current'. Only real changes are sent
Client	<code>pickFrom(ASSET_STATUSES, optional)</code>	Only 1–5 or Enter accepted, so status is always a valid enum value
Client	<code>changes.length() == 0</code>	Short-circuits: no network call
Client	<code>encode(tag)</code>	Percent-encodes %, space, &, ? ... so the tag is URL-safe
Client	GET before PATCH	Fails fast with 404 before asking for input
Server	Payload binding to closed <code>AssetPatch</code>	Unknown fields or a bad status → 400
Server	<code>dateAcquired</code> format, institution registered	400 <code>VALIDATION_FAILED</code> (not used by this client)
Server	Tag exists (inside lock)	404 <code>NOT_FOUND</code>

Gaps worth knowing (good for a viva or report):

- Server-side `patchAsset` does not reject an empty name. Unlike `POST/PUT (validateAsset)`, it accepts `{"name": ""}`. This client can't send that, but another client (curl, the web UI) could.
- Status changes skip business rules. For example, PATCH can set `AVAILABLE` while `currentLoan` is still set, or `LOANED_OUT` with no loan. The loan/return endpoints are the consistent way to change those states.
- `site` isn't checked against the institution's registered sites.
- No optimistic locking (see section 7).